Contents

Python Complete Reference **1**
Índice Rápido . . . 1
1. Sintaxis Básica . . . 1
2. Tipos de Datos . . . 2
3. Operadores . . . 2
4. Strings . . . 3
5. Estructuras de Datos . . . 4
6. Control de Flujo . . . 6
7. Funciones . . . 7
8. Clases y OOP . . . 7
9. Archivos y Excepciones . . . 9
10. Módulos Útiles . . . 10
11. Testing . . . 11
12. Buenas Prácticas . . . 12
Cheat Sheets Individuales . . . 13
Referencias . . . 14

Python Complete Reference

Python 3.13+ | Referencia Completa Unificada

Índice Rápido

Sección	Tema
1	Sintaxis Básica
2	Tipos de Datos
3	Operadores
4	Strings
5	Estructuras de Datos
6	Control de Flujo
7	Funciones
8	Clases y OOP
9	Archivos y Excepciones
10	Módulos Útiles
11	Testing
12	Buenas Prácticas

---

1. Sintaxis Básica

Variables y Asignación

```python
# Asignación
name = "Python"
x, y, z = 1, 2, 3
a = b = c = 0

# Walrus operator
if (n := len(data)) > 10:
    print(f"Large: {n}")
```

```python
# Type hints
name: str = "Alice"
age: int = 30
items: list[str] = []
```

Convenciones de Nombres

Tipo	Convención	Ejemplo
Variables	`snake_case`	`user_name`
Constantes	`UPPER_CASE`	`MAX_SIZE`
Clases	`PascalCase`	`UserAccount`
Funciones	`snake_case`	`get_user()`
Privados	`_prefijo`	`_internal`

---

2. Tipos de Datos

Tipos Primitivos

Tipo	Ejemplo	Mutable
`int`	`42`, `0xFF`	
`float`	`3.14`, `1e-5`	
`bool`	`True`, `False`	
`str`	`"hello"`	
`None`	`None`	

Conversiones

```python
int("42")           # 42
float("3.14")       # 3.14
str(42)             # "42"
bool(1)             # True
list("abc")         # ['a', 'b', 'c']
tuple([1, 2])       # (1, 2)
set([1, 1, 2])      # {1, 2}
dict([("a", 1)])    # {"a": 1}
```

Verificación de Tipos

```python
type(obj)                    # <class 'int'>
isinstance(obj, int)         # True
isinstance(obj, (int, str))  # True si es int O str
```

---

3. Operadores

Aritméticos

Op	Descripción	Ejemplo
`+`	Suma	`5 + 3` → `8`

Op	Descripción	Ejemplo
`-`	Resta	`5 - 3` → `2`
`*`	Multiplicación	`5 * 3` → `15`
`/`	División	`7 / 2` → `3.5`
`//`	División entera	`7 // 2` → `3`
`%`	Módulo	`7 % 2` → `1`
`**`	Potencia	`2 ** 3` → `8`

Comparación

Op	Descripción
`==`	Igual
`!=`	Diferente
`< >`	Menor/Mayor
`<= >=`	Menor/Mayor o igual
`is`	Mismo objeto
`in`	Pertenece a

Lógicos

```
True and False       # False
True or False        # True
not True             # False

# Short-circuit
x = None
result = x or "default"  # "default"
```

---

4. Strings

f-Strings

```
name = "Alice"
f"Hello, {name}!"            # "Hello, Alice!"
f"{3.14159:.2f}"             # "3.14"
f"{1000:,}"                  # "1,000"
f"{'text':>10}"              # "      text"
f"{value=}"                  # "value=42" (debug)
```

Métodos Comunes

```
s = "  Hello, World!  "

# Transformación
s.strip()                    # "Hello, World!"
s.lower()                    # "  hello, world!  "
s.upper()                    # "  HELLO, WORLD!  "
s.replace("World", "Python")

# Búsqueda
s.find("World")              # 9
s.startswith("  Hello")      # True
```

```python
"World" in s                # True

# División/Unión
"a,b,c".split(",")          # ["a", "b", "c"]
",".join(["a", "b", "c"])   # "a,b,c"
```

Slicing

```python
s = "Python"
s[0]        # "P"
s[-1]       # "n"
s[1:4]      # "yth"
s[:3]       # "Pyt"
s[::2]      # "Pto"
s[::-1]     # "nohtyP"
```

---

5. Estructuras de Datos

Resumen

Tipo	Ordenada	Mutable	Duplicados	Sintaxis
`list`				`[1, 2, 3]`
`tuple`				`(1, 2, 3)`
`dict`	*		Keys:	`{"a": 1}`
`set`				`{1, 2, 3}`

Listas

```python
lst = [1, 2, 3]

# Agregar
lst.append(4)               # [1, 2, 3, 4]
lst.extend([5, 6])          # [1, 2, 3, 4, 5, 6]
lst.insert(0, 0)            # [0, 1, 2, 3, 4, 5, 6]

# Eliminar
lst.remove(0)               # Primera ocurrencia
lst.pop()                   # Último
lst.pop(0)                  # Por índice

# Búsqueda
lst.index(3)                # Índice
lst.count(3)                # Ocurrencias
3 in lst                    # True/False

# Ordenar
lst.sort()                  # In-place
lst.sort(reverse=True)
sorted(lst)                 # Nueva lista
```

Diccionarios

```python
d = {"a": 1, "b": 2}

# Acceso
d["a"]                      # 1 (KeyError si no existe)
d.get("c")                  # None
d.get("c", 0)               # 0 (default)

# Modificar
d["c"] = 3                  # Agregar/Actualizar
d.update({"d": 4})
d |= {"e": 5}               # Python 3.9+

# Eliminar
del d["a"]
d.pop("b")

# Iterar
for key in d:
for value in d.values():
for key, value in d.items():

# Merge
merged = d1 | d2            # Python 3.9+
```

Sets

```python
s = {1, 2, 3}

# Operaciones
s.add(4)
s.remove(4)                 # Error si no existe
s.discard(4)                # Sin error

# Operaciones de conjuntos
a | b                       # Unión
a & b                       # Intersección
a - b                       # Diferencia
a ^ b                       # Diferencia simétrica
a <= b                      # Subconjunto
```

Comprensiones

```python
# List
[x**2 for x in range(10)]
[x for x in data if x > 0]

# Dict
{x: x**2 for x in range(5)}
{k: v for k, v in d.items() if v > 0}

# Set
{len(w) for w in words}

# Generator
(x**2 for x in range(10))
```

---

6. Control de Flujo

Condicionales

```
if x > 0:
    print("Positivo")
elif x < 0:
    print("Negativo")
else:
    print("Cero")

# Ternario
result = "par" if x % 2 == 0 else "impar"

# Match (3.10+)
match command:
    case "start":
        start()
    case "stop" | "quit":
        stop()
    case _:
        unknown()
```

Bucles

```
# for
for item in collection:
    process(item)

for i, item in enumerate(collection):
    print(f"{i}: {item}")

for a, b in zip(list1, list2):
    print(a, b)

# while
while condition:
    do_something()

# Control
break           # Salir del bucle
continue        # Siguiente iteración
else:           # Si NO hubo break
```

range()

```
range(5)            # 0, 1, 2, 3, 4
range(2, 8)         # 2, 3, 4, 5, 6, 7
range(0, 10, 2)     # 0, 2, 4, 6, 8
range(10, 0, -1)    # 10, 9, 8, ..., 1
```

---

7. Funciones

Definición

```python
def greet(name: str, greeting: str = "Hello") -> str:
    """Documentación de la función."""
    return f"{greeting}, {name}!"

# Lambda
square = lambda x: x ** 2
```

Parámetros

```python
def func(pos, /, standard, *, kw_only):
    pass

# pos: solo posicional
# standard: posicional o keyword
# kw_only: solo keyword

def func(*args, **kwargs):
    pass

# *args: tupla de argumentos posicionales
# **kwargs: dict de argumentos keyword
```

Decoradores

```python
def decorator(func):
    def wrapper(*args, **kwargs):
        print("Before")
        result = func(*args, **kwargs)
        print("After")
        return result
    return wrapper

@decorator
def my_function():
    pass
```

---

8. Clases y OOP

Clase Básica

```python
class User:
    class_attr = "shared"           # Atributo de clase

    def __init__(self, name: str):
        self.name = name            # Atributo de instancia

    def greet(self) -> str:         # Método de instancia
        return f"Hi, {self.name}"

    @classmethod
```

```python
    def create(cls) -> "User":      # Método de clase
        return cls("Anonymous")

    @staticmethod
    def validate(name: str) -> bool: # Método estático
        return len(name) > 0

    @property
    def display_name(self) -> str:  # Propiedad
        return self.name.title()
```

Herencia

```python
class Animal:
    def speak(self) -> str:
        return "..."

class Dog(Animal):
    def speak(self) -> str:
        return "Woof!"

# Múltiple
class Duck(Animal, Flyable, Swimmable):
    pass
```

Métodos Especiales

```python
__init__(self)          # Constructor
__str__(self)           # str(obj), print(obj)
__repr__(self)          # repr(obj)
__eq__(self, other)     # obj == other
__lt__(self, other)     # obj < other
__len__(self)           # len(obj)
__getitem__(self, key)  # obj[key]
__setitem__(self, k, v) # obj[key] = value
__iter__(self)          # for x in obj
__call__(self)          # obj()
__enter__/__exit__      # with obj
```

Dataclass

```python
from dataclasses import dataclass

@dataclass
class Point:
    x: float
    y: float
    label: str = ""

@dataclass(frozen=True)  # Inmutable
class Config:
    host: str
    port: int
```

---

9. Archivos y Excepciones

Archivos

```python
# Lectura
with open("file.txt", "r", encoding="utf-8") as f:
    content = f.read()
    lines = f.readlines()
    for line in f:
        process(line)

# Escritura
with open("file.txt", "w", encoding="utf-8") as f:
    f.write("Hello\n")
    f.writelines(lines)

# Modos: r, w, a, x, r+, w+, a+, rb, wb
```

pathlib

```python
from pathlib import Path

p = Path("folder") / "file.txt"
p.exists()
p.is_file()
p.read_text(encoding="utf-8")
p.write_text("content", encoding="utf-8")
p.mkdir(parents=True, exist_ok=True)
list(p.glob("*.txt"))
```

JSON

```python
import json

# Leer
with open("data.json", "r") as f:
    data = json.load(f)

# Escribir
with open("data.json", "w") as f:
    json.dump(data, f, indent=2)

# Strings
json.loads('{"key": "value"}')
json.dumps({"key": "value"})
```

Excepciones

```python
try:
    risky_operation()
except ValueError as e:
    handle_error(e)
except (TypeError, KeyError):
    handle_multiple()
except Exception as e:
    handle_any(e)
```

```python
else:
    # Si NO hubo excepción
    success()
finally:
    # SIEMPRE
    cleanup()

# Lanzar
raise ValueError("mensaje")

# Personalizada
class MyError(Exception):
    pass
```

Context Managers

```python
# Usar
with open("file.txt") as f:
    content = f.read()

# Crear con clase
class MyContext:
    def __enter__(self):
        return self
    def __exit__(self, exc_type, exc_val, exc_tb):
        return False

# Crear con decorador
from contextlib import contextmanager

@contextmanager
def my_context():
    setup()
    try:
        yield resource
    finally:
        cleanup()
```

---

10. Módulos Útiles

collections

```python
from collections import Counter, defaultdict, namedtuple, deque

Counter("abracadabra")      # {'a': 5, 'b': 2, ...}
defaultdict(list)           # Dict con valor por defecto
namedtuple("Point", "x y")  # Tupla con nombres
deque([1, 2, 3])            # Lista con O(1) en ambos extremos
```

itertools

```python
from itertools import chain, combinations, permutations, groupby

chain([1, 2], [3, 4])       # 1, 2, 3, 4
```

```
combinations("ABC", 2)       # AB, AC, BC
permutations("AB")           # AB, BA
groupby(data, key=func)      # Agrupar consecutivos
```

functools

```
from functools import reduce, partial, lru_cache

reduce(lambda a, b: a + b, [1, 2, 3])  # 6
partial(func, arg1)                     # Función con args prefijados
@lru_cache(maxsize=128)                 # Memoización
```

datetime

```
from datetime import datetime, date, timedelta

now = datetime.now()
today = date.today()
delta = timedelta(days=7)

now.strftime("%Y-%m-%d %H:%M")
datetime.strptime("2026-01-02", "%Y-%m-%d")
```

re (Regex)

```
import re

re.match(r"^\d+", "123abc")    # Match al inicio
re.search(r"\d+", "abc123")    # Buscar en cualquier lugar
re.findall(r"\d+", "1 and 2")  # Lista de matches
re.sub(r"\d", "X", "a1b2")     # Reemplazar -> "aXbX"
re.split(r"\s+", "a  b   c")   # Split -> ["a", "b", "c"]
```

---

11. Testing

pytest Básico

```
# test_example.py
def test_add():
    assert 1 + 1 == 2

def test_exception():
    import pytest
    with pytest.raises(ValueError):
        int("invalid")
```

Fixtures

```
import pytest

@pytest.fixture
def sample_data():
    return [1, 2, 3]
```

```python
def test_sum(sample_data):
    assert sum(sample_data) == 6
```

Parametrización

```python
@pytest.mark.parametrize("input,expected", [
    (1, 1),
    (2, 4),
    (3, 9),
])
def test_square(input, expected):
    assert input ** 2 == expected
```

Mocking

```python
from unittest.mock import Mock, patch

@patch("module.function")
def test_with_mock(mock_func):
    mock_func.return_value = "mocked"
    result = module.function()
    mock_func.assert_called_once()
```

CLI

```bash
pytest                  # Todos los tests
pytest -v               # Verbose
pytest -x               # Parar en primer fallo
pytest -k "name"        # Por nombre
pytest --cov=src        # Con cobertura
```

---

12. Buenas Prácticas

Hacer

```python
# Type hints
def get_user(user_id: int) -> User | None:

# f-strings
f"Hello, {name}!"

# Context managers
with open("file.txt") as f:

# Comprensiones legibles
[x for x in data if x > 0]

# Early return
def process(data):
    if not data:
        return None
    # ... proceso principal
```

```python
# Excepciones específicas
except ValueError as e:

# Docstrings
def func():
    """Descripción breve."""
```

Evitar

```python
# Sin type hints
def get_user(user_id):

# format() o %
"Hello, {}".format(name)

# except genérico
except:
except Exception:

# Mutables como default
def func(items=[]):  # BUG!

# Variables de una letra
x = get_user()  # ¿qué es x?

# Código profundamente anidado
if a:
    if b:
        if c:
            ...
```

Estructura de Proyecto

```
project/
 src/
    __init__.py
    main.py
    models/
    services/
    utils/
 tests/
    __init__.py
    conftest.py
    test_*.py
 pyproject.toml
 README.md
```

Cheat Sheets Individuales

Para información más detallada, consulta los cheat sheets específicos:

1. **Python Basics** - Variables, tipos, operadores, strings
2. **Data Structures** - Listas, diccionarios, sets, tuplas
3. **OOP Python** - Clases, herencia, patrones
4. **File Handling** - Archivos, excepciones, context managers

5. **Testing & Debugging** - pytest, mocking, logging

Referencias

- **Python Docs**: docs.python.org
- **PEP 8**: pep8.org
- **pytest**: docs.pytest.org
- **Real Python**: realpython.com

Complete Reference para el Bootcamp Python Zero to Hero - 2026